

Dependency Injection e Factory Patterns em .NET

Material Teórico Completo para Ensino de Padrões de Criação

Versão: 1.0

Framework Alvo: .NET 8 (LTS)

Nível: Intermediário/Avançado

Carga Horária: 8 horas-aula (4 encontros)

Público: Alunos de graduação em Computação com conhecimento de C#, OOP, interfaces e generics

Sumário Executivo

Este documento apresenta uma abordagem integrada para o ensino de Dependency Injection (DI) e Factory Patterns no contexto de aplicações empresariais .NET modernas. O material foi estruturado para demonstrar como esses padrões se complementam na construção de arquiteturas extensíveis, testáveis e que respeitam os princípios SOLID.

MÓDULO 1: Fundamentos de Dependency Injection

1.1 O Problema do Acoplamento Forte

Antes de compreendermos Dependency Injection, precisamos entender o problema que ele resolve. Considere um serviço típico em uma aplicação:

```
csharp
```

// **✗ PROBLEMA:** Acoplamento forte e baixa testabilidade

```
public class OrderService
{
    private readonly EmailNotifier _notifier;
    private readonly SqlOrderRepository _repository;

    public OrderService()
    {
        _notifier = new EmailNotifier(); // Criação direta = acoplamento
        _repository = new SqlOrderRepository(); // Dependência concreta
    }

    public void ProcessOrder(Order order)
    {
        _repository.Save(order);
        _notifier.SendConfirmation(order.CustomerEmail);
    }
}
```

Problemas identificados neste código:

Primeiro, há acoplamento direto às implementações concretas. A classe `OrderService` conhece e instancia diretamente `EmailNotifier` e `SqlOrderRepository`, criando dependências rígidas que dificultam mudanças futuras.

Segundo, a testabilidade é severamente comprometida. Para testar `OrderService`, somos forçados a executar código real de envio de e-mail e acesso ao banco de dados, tornando os testes lentos, frágeis e dependentes de infraestrutura externa.

Terceiro, há violação do princípio de responsabilidade única. `OrderService` não apenas processa pedidos, mas também gerencia a criação de suas próprias dependências, assumindo múltiplas responsabilidades.

Quarto, a extensibilidade é limitada. Se precisarmos trocar o mecanismo de notificação (por exemplo, de e-mail para SMS) ou o repositório (de SQL para NoSQL), precisamos modificar o código fonte de `OrderService`, violando o princípio Open/Closed.

1.2 Inversão de Controle (IoC) - O Princípio Fundamental

Inversão de Controle é um princípio de design que inverte o fluxo de controle tradicional de um programa. Em vez de o código da aplicação controlar a criação e o ciclo de vida de objetos, essa responsabilidade é delegada a um componente externo, tipicamente chamado de container ou framework.

Conceito-chave: "Não nos chame, nós chamaremos você" (Hollywood Principle).

Na abordagem tradicional, seu código solicita ativamente as dependências que precisa. Com IoC, você declara quais dependências precisa, e um componente externo é responsável por fornecer essas dependências quando necessário.

1.3 Dependency Injection como Implementação de IoC

Dependency Injection é um padrão específico de implementação do princípio IoC. A ideia central é que as dependências de uma classe sejam fornecidas externamente (injetadas) em vez de criadas internamente pela própria classe.

Existem três formas principais de injeção:

Constructor Injection (Injeção por Construtor) - Esta é a forma mais recomendada e amplamente utilizada. As dependências são fornecidas através do construtor da classe, tornando explícitas todas as dependências obrigatórias e garantindo que o objeto nunca exista em estado inválido.

```
csharp
// ✅ BOM: Dependências injetadas via construtor
public class OrderService
{
    private readonly INotifier _notifier;
    private readonly IOrderRepository _repository;

    // Dependências são parâmetros do construtor
    public OrderService(INotifier notifier, IOrderRepository repository)
    {
        _notifier = notifier ?? throw new ArgumentNullException(nameof(notifier));
        _repository = repository ?? throw new ArgumentNullException(nameof(repository));
    }

    public void ProcessOrder(Order order)
    {
        _repository.Save(order);
        _notifier.SendConfirmation(order.CustomerEmail);
    }
}
```

Property Injection (Injeção por Propriedade) - As dependências são configuradas através de propriedades públicas após a construção do objeto. Esta abordagem é menos comum e geralmente reservada para dependências opcionais.

Method Injection (Injeção por Método) - As dependências são passadas como parâmetros para métodos específicos. Útil quando a dependência é necessária apenas para uma operação específica e não para todo o ciclo de vida do objeto.

1.4 Microsoft.Extensions.DependencyInjection

O .NET fornece um container de DI nativo e leve através do pacote `Microsoft.Extensions.DependencyInjection`. Este container é o padrão utilizado em ASP.NET Core, mas pode ser usado em qualquer tipo de aplicação .NET (console, desktop, serviços, etc.).

Componentes principais:

IServiceCollection é a interface que representa a coleção de descritores de serviços. É aqui que você registra todos os serviços da sua aplicação, definindo como cada tipo deve ser resolvido.

IServiceProvider é a interface que representa o container configurado e permite a resolução de serviços. É através dele que você solicita instâncias dos tipos registrados.

ServiceDescriptor é a classe que descreve um serviço registrado, incluindo o tipo de serviço, o tipo de implementação e o tempo de vida (lifetime).

1.5 Lifetimes: Gerenciamento do Ciclo de Vida

Um dos aspectos mais importantes da DI é o gerenciamento do ciclo de vida dos objetos. O .NET oferece três lifetimes principais:

Transient - Uma nova instância é criada cada vez que o serviço é solicitado. Este é o lifetime mais simples e adequado para serviços leves e stateless (sem estado).

```
csharp
services.AddTransient<INotifier, EmailNotifier>();
// Cada resolução cria uma nova instância
```

Use Transient quando o serviço não mantém estado entre chamadas, é leve para construir, e você não precisa compartilhar a mesma instância entre diferentes consumidores.

Scoped - Uma única instância é criada por escopo (scope). Em aplicações web, um escopo geralmente corresponde a uma requisição HTTP. A mesma instância é compartilhada dentro daquele escopo, mas escopos diferentes têm instâncias diferentes.

```
csharp
services.AddScoped<IOrderRepository, SqlOrderRepository>();
// Mesma instância durante toda a requisição HTTP
```

Use Scoped para serviços que mantêm estado relacionado a uma operação específica (como contextos de banco de dados), onde você quer garantir consistência durante uma operação lógica, mas não quer compartilhar estado entre operações diferentes.

Singleton - Uma única instância é criada para toda a vida da aplicação. A primeira resolução cria a instância, e todas as resoluções subsequentes retornam a mesma instância.

```
csharp
services.AddSingleton<IConfigurationService, ConfigurationService>();
// Uma única instância para toda a aplicação
```

Use Singleton para serviços que são caros para construir, são thread-safe, mantêm estado que deve ser compartilhado globalmente, ou representam recursos únicos (como configurações).

Regra Crítica de Segurança: Nunca injete um serviço Transient ou Scoped em um Singleton. Isso cria um "captive dependency" onde o serviço de vida curta fica preso no Singleton, causando vazamentos de memória e comportamentos inesperados.

1.6 Padrões de Registro no IServiceCollection

O .NET oferece métodos convenientes para diferentes cenários de registro:

```
csharp

// Registro básico: Interface → Implementação
services.AddTransient<INotifier, EmailNotifier>();

// Registro de implementação concreta (sem interface)
services.AddTransient<EmailService>();

// Registro com factory method (para lógica de criação complexa)
services.AddTransient<INotifier>(provider =>
{
    var config = provider.GetRequiredService<IConfiguration>();
    var smtpServer = config["Email:SmtpServer"];
    return new EmailNotifier(smtpServer);
});

// Registro de múltiplas implementações da mesma interface
services.AddTransient<INotifier, EmailNotifier>();
services.AddTransient<INotifier, SmsNotifier>();
services.AddTransient<INotifier, PushNotifier>();

// Registro de instância existente (sempre Singleton)
var existingLogger = new FileLogger("app.log");
services.AddSingleton<ILogger>(existingLogger);

// Registro genérico
services.AddTransient(typeof(IRepository<>), typeof(Repository<>));
```

1.7 Resolução de Serviços e Injeção Automática

Uma vez registrados os serviços, o container resolve automaticamente as dependências:

```
csharp
```

```

// No método ConfigureServices (Startup ou Program.cs)
services.AddTransient<INotifier, EmailNotifier>();
services.AddScoped<IOrderRepository, SqlOrderRepository>();
services.AddScoped<OrderService>();

// Em um controlador ou outro serviço
public class OrderController : ControllerBase
{
    private readonly OrderService _orderService;

    // O container injeta automaticamente OrderService
    // que por sua vez recebe INotifier e IOrderRepository
    public OrderController(OrderService orderService)
    {
        _orderService = orderService;
    }
}

```

O container resolve recursivamente toda a árvore de dependências. Se `OrderService` precisa de `INotifier` e `IOrderRepository`, e `EmailNotifier` precisa de `IConfiguration`, o container resolve tudo automaticamente, respeitando os lifetimes configurados.

1.8 Testabilidade com DI

Um dos maiores benefícios da DI é a testabilidade. Com dependências injetadas através de interfaces, podemos facilmente substituí-las por implementações de teste (mocks ou stubs):

```
csharp
```

// Teste unitário com xUnit e Moq

public class OrderServiceTests

{

[Fact]

public void ProcessOrder_ShouldSaveAndNotify()

{

// Arrange: Criamos mocks das dependências

var mockNotifier = **new** Mock<INotifier>();

var mockRepository = **new** Mock<IOrderRepository>();

// Injetamos os mocks no serviço

var service = **new** OrderService(

mockNotifier.Object,

mockRepository.Object

);

var order = **new** Order { Id = 1, CustomerEmail = "test@example.com" };

// Act: Executamos o método sendo testado

service.ProcessOrder(order);

// Assert: Verificamos que os métodos foram chamados corretamente

mockRepository.Verify(r => r.Save(order), Times.Once);

mockNotifier.Verify(n => n.SendConfirmation("test@example.com"), Times.Once);

}

}

1.9 Anti-Pattern: Service Locator

Um erro comum ao aprender DI é usar o container como Service Locator:

csharp

// ❌ ANTI-PATTERN: Service Locator

```
public class OrderService
{
    private readonly IServiceProvider _serviceProvider;

    public OrderService(IServiceProvider serviceProvider)
    {
        _serviceProvider = serviceProvider;
    }

    public void ProcessOrder(Order order)
    {
        // Solicitando dependências sob demanda
        var notifier = _serviceProvider.GetRequiredService<INotifier>();
        var repository = _serviceProvider.GetRequiredService<IOrderRepository>();

        repository.Save(order);
        notifier.SendConfirmation(order.CustomerEmail);
    }
}
```

Por que isso é problemático:

As dependências da classe não ficam explícitas. Olhando apenas a assinatura do construtor, não sabemos que `OrderService` precisa de `INotifier` e `IOrderRepository`.

Dificulta os testes, pois precisamos criar um `IServiceProvider` completo mesmo para testes simples.

Viola o princípio de inversão de dependência, pois a classe conhece o mecanismo de resolução (o container) em vez de apenas suas dependências.

Exceção: O uso de `IServiceProvider` é aceitável em factories e em pontos de extensibilidade onde você precisa resolver tipos dinamicamente em runtime.

1.10 Integração com Generic Host

Para aplicações console e serviços, o .NET oferece o Generic Host que fornece infraestrutura completa de DI, configuração e logging:

```
csharp
```



```
using Microsoft.Extensions.Hosting;
using Microsoft.Extensions.DependencyInjection;

// Program.cs
var host = Host.CreateDefaultBuilder(args)
    .ConfigureServices((context, services) =>
    {
        // Registre seus serviços aqui
        services.AddTransient<INotifier, EmailNotifier>();
        services.AddScoped<IOrderRepository, SqlOrderRepository>();
        services.AddScoped<OrderService>();

        // Registre o serviço principal que será executado
        services.AddHostedService<ApplicationService>();
    })
    .Build();

await host.RunAsync();

// ApplicationService.cs - Serviço principal da aplicação
public class ApplicationService : IHostedService
{
    private readonly OrderService _orderService;

    public ApplicationService(OrderService orderService)
    {
        _orderService = orderService;
    }

    public Task StartAsync(CancellationToken cancellationToken)
    {
        // Lógica de inicialização
        return Task.CompletedTask;
    }

    public Task StopAsync(CancellationToken cancellationToken)
    {
        // Lógica de encerramento
        return Task.CompletedTask;
    }
}
```

MÓDULO 2: Factory Patterns - Fundamentos

2.1 O Conceito de Factory

Factory (Fábrica) é um termo genérico para padrões de criação que encapsulam a lógica de instanciação de objetos. O objetivo fundamental é separar o código que usa objetos do código que cria objetos.

Motivações para usar Factories:

A lógica de criação é complexa e não deve poluir o código cliente. Por exemplo, criar um objeto pode envolver múltiplas etapas, validações, configurações ou dependências que precisam ser resolvidas.

O tipo concreto a ser criado depende de condições de runtime, como valores de configuração, dados do usuário ou estado da aplicação.

Você quer garantir que objetos sejam criados de forma consistente, seguindo regras de negócio ou restrições técnicas específicas.

O código cliente não deve conhecer todas as classes concretas disponíveis, promovendo baixo acoplamento e aderência ao princípio de inversão de dependência.

2.2 Simple Factory (Factory Estático)

O Simple Factory não é um padrão GoF (Gang of Four) oficial, mas é uma técnica comum e útil. Consiste em uma classe ou método estático que encapsula a criação de objetos:

```
csharp

// Simple Factory
public static class NotifierFactory
{
    public static INotifier Create(string type)
    {
        return type.ToLower() switch
        {
            "email" => new EmailNotifier(),
            "sms" => new SmsNotifier(),
            "push" => new PushNotifier(),
            _ => throw new ArgumentException($"Tipo desconhecido: {type}")
        };
    }
}

// Uso
var notifier = NotifierFactory.Create("email");
notifier.SendConfirmation("user@example.com");
```

Vantagens: Simples de implementar e entender. Centraliza a lógica de criação em um único lugar. Reduz duplicação de código de criação.

Limitações: O factory precisa conhecer todas as implementações concretas, violando o princípio Open/Closed. Difícil de estender sem modificar o código do factory. Não integra bem com DI, pois usa instanciamento direto. Dificulta testes, pois você não pode facilmente substituir o comportamento do factory.

Quando usar: Para cenários simples onde o conjunto de tipos é fixo e bem conhecido. Como ponto de partida antes de evoluir para padrões mais sofisticados. Em utilitários ou helpers onde simplicidade é mais importante que extensibilidade.

2.3 Factory Method Pattern

O Factory Method é um padrão GoF que define uma interface para criar objetos, mas permite que subclasses decidam qual classe instanciar. Ele delega a responsabilidade de instanciamento para subclasses.

Estrutura do padrão:

csharp

// Produto abstrato

```
public interface INotifier
{
    void SendConfirmation(string recipient);
}
```

// Produtos concretos

```
public class EmailNotifier : INotifier
{
    public void SendConfirmation(string recipient)
    {
        Console.WriteLine($"Email enviado para {recipient}");
    }
}
```

```
public class SmsNotifier : INotifier
{
    public void SendConfirmation(string recipient)
    {
        Console.WriteLine($"SMS enviado para {recipient}");
    }
}
```

// Creator abstrato com Factory Method

```
public abstract class NotificationService
{
    // Factory Method - subclasses implementam
    protected abstract INotifier CreateNotifier();

    // Método template que usa o Factory Method
    public void NotifyCustomer(string recipient)
    {
        var notifier = CreateNotifier();
        notifier.SendConfirmation(recipient);
    }
}
```

// Creators concretos

```
public class EmailNotificationService : NotificationService
{
    protected override INotifier CreateNotifier()
    {
        return new EmailNotifier();
    }
}
```

```
public class SmsNotificationService : NotificationService
{
    protected override INotifier CreateNotifier()
    {
        return new SmsNotifier();
    }
}
```

Uso:

```
csharp

NotificationService service = new EmailNotificationService();
service.NotifyCustomer("user@example.com");
```

Vantagens: Segue o princípio Open/Closed - você pode adicionar novos tipos sem modificar código existente. Encapsula a lógica de criação em locais apropriados. Promove baixo acoplamento entre código cliente e classes concretas.

Quando usar: Quando a classe não pode antecipar o tipo de objetos que deve criar. Quando você quer que subclasses especifiquem os objetos a serem criados. Quando há lógica complexa que precede a criação do objeto.

2.4 Abstract Factory Pattern

O Abstract Factory fornece uma interface para criar famílias de objetos relacionados sem especificar suas classes concretas. É particularmente útil quando você precisa garantir que objetos criados sejam compatíveis entre si.

Cenário motivador: Imagine um sistema de geração de relatórios que pode produzir relatórios em diferentes formatos (PDF, Excel, HTML), e cada formato tem componentes específicos (cabeçalho, tabela, gráfico) que devem ser consistentes.

```
csharp
```

```
// Produtos abstratos - componentes de relatório
```

```
public interface IReportHeader
```

```
{  
    void Render(string title);  
}
```

```
public interface IReportTable
```

```
{  
    void Render(string[][] data);  
}
```

```
public interface IReportChart
```

```
{  
    void Render(double[] values);  
}
```

```
// Produtos concretos - Família PDF
```

```
public class PdfReportHeader : IReportHeader
```

```
{  
    public void Render(string title)  
    {  
        Console.WriteLine($"[PDF] Renderizando cabeçalho: {title}");  
    }  
}
```

```
public class PdfReportTable : IReportTable
```

```
{  
    public void Render(string[][] data)  
    {  
        Console.WriteLine($"[PDF] Renderizando tabela com {data.Length} linhas");  
    }  
}
```

```
public class PdfReportChart : IReportChart
```

```
{  
    public void Render(double[] values)  
    {  
        Console.WriteLine($"[PDF] Renderizando gráfico com {values.Length} pontos");  
    }  
}
```

```
// Produtos concretos - Família Excel
```

```
public class ExcelReportHeader : IReportHeader
```

```
{  
    public void Render(string title)  
    {  
        Console.WriteLine($"[Excel] Renderizando cabeçalho: {title}");  
    }  
}
```

```
        Console.WriteLine($"[Excel] Renderizando cabeçalho: {title}");
    }
}

public class ExcelReportTable : IReportTable
{
    public void Render(string[][] data)
    {
        Console.WriteLine($"[Excel] Renderizando tabela com {data.Length} linhas");
    }
}

public class ExcelReportChart : IReportChart
{
    public void Render(double[] values)
    {
        Console.WriteLine($"[Excel] Renderizando gráfico com {values.Length} pontos");
    }
}

// Abstract Factory
public interface IReportFactory
{
    IReportHeader CreateHeader();
    IReportTable CreateTable();
    IReportChart CreateChart();
}

// Concrete Factories
public class PdfReportFactory : IReportFactory
{
    public IReportHeader CreateHeader() => new PdfReportHeader();
    public IReportTable CreateTable() => new PdfReportTable();
    public IReportChart CreateChart() => new PdfReportChart();
}

public class ExcelReportFactory : IReportFactory
{
    public IReportHeader CreateHeader() => new ExcelReportHeader();
    public IReportTable CreateTable() => new ExcelReportTable();
    public IReportChart CreateChart() => new ExcelReportChart();
}

// Código cliente
public class ReportGenerator
{
    private readonly IReportFactory _factory;
```

```

public ReportGenerator(IReportFactory factory)
{
    _factory = factory;
}

public void GenerateReport(string title, string[][] data, double[] chartData)
{
    var header = _factory.CreateHeader();
    var table = _factory.CreateTable();
    var chart = _factory.CreateChart();

    header.Render(title);
    table.Render(data);
    chart.Render(chartData);
}
}

```

Uso:

```

csharp

// Cliente trabalha apenas com interfaces
IReportFactory factory = new PdfReportFactory();
var generator = new ReportGenerator(factory);
generator.GenerateReport("Vendas Q4", salesData, chartData);

// Trocar para Excel é trivial
factory = new ExcelReportFactory();
generator = new ReportGenerator(factory);
generator.GenerateReport("Vendas Q4", salesData, chartData);

```

Vantagens: Garante que produtos de uma família sejam usados juntos. Isola código cliente de classes concretas. Facilita a troca de famílias inteiras de produtos. Promove consistência entre produtos relacionados.

Quando usar: Quando o sistema precisa ser independente de como seus produtos são criados. Quando você precisa trabalhar com múltiplas famílias de produtos relacionados. Quando você quer garantir que produtos de uma família sejam usados juntos.

MÓDULO 3: Integrando Factories com Dependency Injection

3.1 O Problema: Factories Tradicionais vs DI

Os padrões Factory clássicos foram criados antes da popularização de containers de DI. Quando implementados

literalmente, podem entrar em conflito com os princípios de DI:

csharp

```
// ❌ PROBLEMA: Factory cria instâncias diretamente
public class TraditionalPaymentFactory : IPaymentFactory
{
    public IPaymentProcessor Create(string type)
    {
        return type switch
        {
            "credit" => new CreditCardProcessor(), // Instanciação direta
            "boleto" => new BoletoProcessor(),     // Sem DI
            "pix" => new PixProcessor(),           // Dependências não injetadas
            _ => throw new ArgumentException()
        };
    }
}
```

Problemas desta abordagem:

Se `CreditCardProcessor` precisa de dependências (como `ILogger` ou `IConfiguration`), o factory precisa conhecê-las e criá-las, aumentando o acoplamento.

Não há controle sobre o lifecycle dos objetos criados.

Testes ficam complicados, pois não podemos substituir as implementações.

3.2 Solução: Factory + Service Provider

A solução moderna é combinar o padrão Factory com o container de DI:

csharp

```
//  BOM: Factory usa o container para resolver dependências
public class PaymentFactory : IPaymentFactory
{
    private readonly IServiceProvider _serviceProvider;

    public PaymentFactory(IServiceProvider serviceProvider)
    {
        _serviceProvider = serviceProvider;
    }

    public IPaymentProcessor Create(string type)
    {
        return type switch
        {
            "credit" => _serviceProvider.GetRequiredService<CreditCardProcessor>(),
            "boleto" => _serviceProvider.GetRequiredService<BoletoProcessor>(),
            "pix" => _serviceProvider.GetRequiredService<PixProcessor>(),
            _ => throw new ArgumentException($"Tipo de pagamento desconhecido: {type}")
        };
    }
}

// Registro no container
services.AddTransient<CreditCardProcessor>();
services.AddTransient<BoletoProcessor>();
services.AddTransient<PixProcessor>();
services.AddSingleton<IPaymentFactory, PaymentFactory>();
```

Nota importante: Este é um dos poucos casos onde injetar `IServiceProvider` é aceitável, pois o factory é essencialmente um ponto de extensibilidade que precisa resolver tipos dinamicamente.

3.3 Pattern: Strongly-Typed Factory com Generics

Podemos melhorar ainda mais usando generics para criar factories type-safe:

```
csharp
```

```

// Factory genérico
public interface IFactory<T>
{
    T Create();
}

public class Factory<T> : IFactory<T> where T : class
{
    private readonly IServiceProvider _serviceProvider;

    public Factory(IServiceProvider serviceProvider)
    {
        _serviceProvider = serviceProvider;
    }

    public T Create()
    {
        return _serviceProvider.GetRequiredService<T>();
    }
}

// Registro
services.AddTransient(typeof(IFactory<>), typeof(Factory<>));

// Uso
public class PaymentService
{
    private readonly IFactory<CreditCardProcessor> _creditCardFactory;

    public PaymentService(IFactory<CreditCardProcessor> creditCardFactory)
    {
        _creditCardFactory = creditCardFactory;
    }

    public void ProcessCreditCard(Payment payment)
    {
        var processor = _creditCardFactory.Create();
        processor.Process(payment);
    }
}

```

3.4 Pattern: Named Factory com Dictionary

Para cenários onde você precisa selecionar entre múltiplas implementações baseado em uma chave:

// Factory com seleção por nome

```
public interface INamedFactory<T>
```

```
{  
    T Create(string name);  
    IEnumerable<string> GetAvailableNames();  
}
```

```
public class NamedFactory<T> : INamedFactory<T> where T : class
```

```
{  
    private readonly IServiceProvider _serviceProvider;  
    private readonly Dictionary<string, Type> _registrations;
```

```
    public NamedFactory(  
        IServiceProvider serviceProvider,  
        Dictionary<string, Type> registrations)  
    {  
        _serviceProvider = serviceProvider;  
        _registrations = registrations;  
    }
```

```
    public T Create(string name)  
    {  
        if (!_registrations.TryGetValue(name, out var type))  
        {  
            throw new ArgumentException($"Tipo não registrado: {name}");  
        }  
  
        return (T)_serviceProvider.GetRequiredService(type);  
    }
```

```
    public IEnumerable<string> GetAvailableNames()  
    {  
        return _registrations.Keys;  
    }  
}
```

// Registro

```
services.AddTransient<EmailNotifier>();  
services.AddTransient<SmsNotifier>();  
services.AddTransient<PushNotifier>();
```

```
services.AddSingleton<INamedFactory<INotifier>>(provider =>  
{  
    var registrations = new Dictionary<string, Type>  
    {  
        ["email"] = typeof(EmailNotifier),
```

```
["sms"] = typeof(SmsNotifier),  
["push"] = typeof(PushNotifier)  
};  
return new NamedFactory<INotifier>(provider, registrations);  
});
```

3.5 Pattern: Factory com Func<T>

O .NET permite registrar delegates diretamente, criando factories inline:

```
csharp  
  
// Registro de factory como Func<T>  
services.AddTransient<CreditCardProcessor>();  
services.AddTransient<Func<CreditCardProcessor>>>(provider =>  
{  
    return () => provider.GetRequiredService<CreditCardProcessor>();  
});  
  
// Uso - extremamente limpo  
public class PaymentService  
{  
    private readonly Func<CreditCardProcessor> _processorFactory;  
  
    public PaymentService(Func<CreditCardProcessor> processorFactory)  
    {  
        _processorFactory = processorFactory;  
    }  
  
    public void ProcessPayment(Payment payment)  
    {  
        var processor = _processorFactory();  
        processor.Process(payment);  
    }  
}
```

Vantagens deste approach: Sintaxe muito limpa e idiomática em C#. Não requer criação de interfaces ou classes de factory extras. Integra perfeitamente com DI. Fácil de testar - você simplesmente passa um delegate de teste.

3.6 Abstract Factory com DI

Podemos integrar o padrão Abstract Factory com DI de forma elegante:

```
csharp
```

// Abstract Factory

```
public interface IReportFactory
{
    IReportHeader CreateHeader();
    IReportTable CreateTable();
    IReportChart CreateChart();
}
```

// Concrete Factory que usa DI

```
public class PdfReportFactory : IReportFactory
{
    private readonly IServiceProvider _serviceProvider;

    public PdfReportFactory(IServiceProvider serviceProvider)
    {
        _serviceProvider = serviceProvider;
    }

    public IReportHeader CreateHeader()
    => _serviceProvider.GetRequiredService<PdfReportHeader>();

    public IReportTable CreateTable()
    => _serviceProvider.GetRequiredService<PdfReportTable>();

    public IReportChart CreateChart()
    => _serviceProvider.GetRequiredService<PdfReportChart>();
}
```

// Registro

```
services.AddTransient<PdfReportHeader>();
services.AddTransient<PdfReportTable>();
services.AddTransient<PdfReportChart>();
services.AddTransient<IReportFactory, PdfReportFactory>();
```

// Factory selector (escolhe qual factory usar)

```
services.AddSingleton<Func<string, IReportFactory>>(provider =>
{
    return format => format.ToLower() switch
    {
        "pdf" => provider.GetRequiredService<PdfReportFactory>(),
        "excel" => provider.GetRequiredService<ExcelReportFactory>(),
        "html" => provider.GetRequiredService<HtmlReportFactory>(),
        _ => throw new ArgumentException($"Formato desconhecido: {format}")
    };
});
```

MÓDULO 4: Project Factory - Orquestração de Módulos

4.1 Conceito de Project Factory

O termo "Project Factory" não é um padrão GoF clássico, mas refere-se a um padrão arquitetural comum em aplicações empresariais .NET: uma factory responsável por orquestrar a criação, configuração e registro de módulos ou plugins em uma aplicação.

Características principais:

Gerencia o ciclo de vida completo de módulos, desde descoberta até inicialização.

Coordena o registro de serviços de múltiplos módulos no container de DI.

Fornece pontos de extensão para plugins externos.

Mantém metadados sobre módulos disponíveis.

Casos de uso reais:

Aplicações modulares onde funcionalidades podem ser habilitadas/desabilitadas.

Sistemas de plugins onde terceiros podem estender a aplicação.

Arquiteturas de microkernel onde o núcleo é mínimo e funcionalidades são adicionadas via módulos.

Aplicações multi-tenant onde diferentes tenants têm diferentes conjuntos de funcionalidades.

4.2 Anatomia de um Sistema Modular

Um sistema modular típico tem os seguintes componentes:

Module Definition (Definição de Módulo) - Interface que todo módulo deve implementar:

```
csharp
public interface IModule
{
    string Name { get; }
    string Version { get; }
    string Description { get; }

    void ConfigureServices(IServiceCollection services);
    void Initialize(IServiceProvider serviceProvider);
}
```

Module Metadata (Metadados) - Informações sobre dependências e requisitos:

csharp

```
[AttributeUsage(AttributeTargets.Class)]
public class ModuleAttribute : Attribute
{
    public string Name { get; set; }
    public string[] Dependencies { get; set; }
    public bool IsOptional { get; set; }
}
```

Module Registry (Registro de Módulos) - Mantém informações sobre módulos disponíveis:

csharp

```
public interface IModuleRegistry
{
    void Register<TModule>() where TModule : IModule;
    IEnumerable<ModuleInfo> GetRegisteredModules();
    ModuleInfo GetModule(string name);
}
```

Module Loader (Carregador) - Descobre e carrega módulos:

csharp

```
public interface IModuleLoader
{
    IEnumerable<IModule> DiscoverModules();
    IModule LoadModule(string assemblyPath);
}
```

Project Factory (Orquestrador) - Coordena todo o processo:

csharp

```
public interface IProjectFactory
{
    void BuildProject(IServiceCollection services);
    void InitializeModules(IServiceProvider serviceProvider);
}
```

4.3 Implementação de Project Factory

Vamos implementar um Project Factory completo:

csharp


```
public class ProjectFactory : IProjectFactory
{
    private readonly IModuleRegistry _registry;
    private readonly IModuleLoader _loader;
    private readonly ILogger<ProjectFactory> _logger;
    private readonly List<IModule> _loadedModules;

    public ProjectFactory(
        IModuleRegistry registry,
        IModuleLoader loader,
        ILogger<ProjectFactory> logger)
    {
        _registry = registry;
        _loader = loader;
        _logger = logger;
        _loadedModules = new List<IModule>();
    }

    public void BuildProject(IServiceCollection services)
    {
        _logger.LogInformation("Iniciando construção do projeto...");

        // 1. Descobrir módulos disponíveis
        var discoveredModules = _loader.DiscoverModules();
        _logger.LogInformation($"Descobertos {discoveredModules.Count()} módulos");

        // 2. Resolver ordem de carregamento (respeitando dependências)
        var orderedModules = ResolveLoadOrder(discoveredModules);

        // 3. Carregar cada módulo
        foreach (var module in orderedModules)
        {
            try
            {
                _logger.LogInformation($"Carregando módulo: {module.Name} v{module.Version}");

                // Registrar serviços do módulo
                module.ConfigureServices(services);

                _loadedModules.Add(module);
                _registry.Register(module);

                _logger.LogInformation($"Módulo {module.Name} carregado com sucesso");
            }
            catch (Exception ex)
            {

```

```

_logger.LogError(ex, $"Erro ao carregar módulo {module.Name}");

// Decidir se continua ou para (baseado em IsOptional)
var metadata = GetModuleMetadata(module);
if (!metadata.IsOptional)
{
    throw new ModuleLoadException($"Falha ao carregar módulo obrigatório: {module.Name}", ex);
}
}

_logger.LogInformation($"Projeto construído com {_loadedModules.Count} módulos");
}

public void InitializeModules(IServiceProvider serviceProvider)
{
    _logger.LogInformation("Iniciando módulos...");

    foreach (var module in _loadedModules)
    {
        try
        {
            _logger.LogInformation($"Iniciando módulo: {module.Name}");
            module.Initialize(serviceProvider);
            _logger.LogInformation($"Módulo {module.Name} inicializado");
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, $"Erro ao inicializar módulo {module.Name}");
            throw;
        }
    }

    _logger.LogInformation("Todos os módulos inicializados");
}

private IEnumerable<IModule> ResolveLoadOrder(IEnumerable<IModule> modules)
{
    // Implementação de ordenação topológica baseada em dependências
    // (Simplificado para brevidade)
    var sorted = new List<IModule>();
    var visited = new HashSet<string>();

    foreach (var module in modules)
    {
        ResolveModule(module, modules, visited, sorted);
    }
}

```

```

        return sorted;
    }

    private void ResolveModule(
        IModule module,
        IEnumerable<IModule> allModules,
        HashSet<string> visited,
        List<IModule> sorted)
    {
        if (visited.Contains(module.Name))
            return;

        visited.Add(module.Name);

        var metadata = GetModuleMetadata(module);
        if (metadata.Dependencies != null)
        {
            foreach (var depName in metadata.Dependencies)
            {
                var dependency = allModules.FirstOrDefault(m => m.Name == depName);
                if (dependency != null)
                {
                    ResolveModule(dependency, allModules, visited, sorted);
                }
            }
        }

        sorted.Add(module);
    }

    private ModuleAttribute GetModuleMetadata(IModule module)
    {
        return module.GetType().GetCustomAttribute<ModuleAttribute>()
            ?? new ModuleAttribute { Name = module.Name, IsOptional = true };
    }
}

```

4.4 Exemplo de Módulo Concreto

csharp

```
[Module(Name = "Notifications", Dependencies = new[] { "Logging" }, IsOptional = false)]
public class NotificationsModule : IModule
{
    public string Name => "Notifications";
    public string Version => "1.0.0";
    public string Description => "Módulo de notificações multi-canal";

    public void ConfigureServices(IServiceCollection services)
    {
        // Registrar serviços do módulo
        services.AddTransient<INotifier, EmailNotifier>();
        services.AddTransient<INotifier, SmsNotifier>();
        services.AddTransient<INotifier, PushNotifier>();

        // Registrar factory para seleção de notifier
        services.AddSingleton<INotificationFactory, NotificationFactory>();

        // Registrar serviço principal do módulo
        services.AddScoped<INotificationService, NotificationService>();
    }

    public void Initialize(IServiceProvider serviceProvider)
    {
        // Inicialização pós-registro (ex: carregar configurações, validar dependências)
        var logger = serviceProvider.GetRequiredService<ILogger<NotificationsModule>>();
        logger.LogInformation("Módulo de notificações inicializado");

        // Validar configuração
        var config = serviceProvider.GetRequiredService<IConfiguration>();
        if (string.IsNullOrEmpty(config["Notifications:SmtpServer"]))
        {
            logger.LogWarning("Servidor SMTP não configurado");
        }
    }
}
```

4.5 Descoberta Dinâmica de Módulos

Para permitir plugins externos, podemos implementar descoberta dinâmica via reflexão:

csharp

```
public class AssemblyModuleLoader : IModuleLoader
{
    private readonly ILogger<AssemblyModuleLoader> _logger;
    private readonly string _modulesPath;

    public AssemblyModuleLoader(
        ILogger<AssemblyModuleLoader> logger,
        IConfiguration configuration)
    {
        _logger = logger;
        _modulesPath = configuration["Modules:Path"] ?? "Modules";
    }

    public IEnumerable<IModule> DiscoverModules()
    {
        var modules = new List<IModule>();

        // Descobrir assemblies no diretório de módulos
        if (!Directory.Exists(_modulesPath))
        {
            _logger.LogWarning($"Diretório de módulos não encontrado: {_modulesPath}");
            return modules;
        }

        var assemblyFiles = Directory.GetFiles(_modulesPath, "*.dll");
        _logger.LogInformation($"Encontrados {assemblyFiles.Length} assemblies");

        foreach (var file in assemblyFiles)
        {
            try
            {
                var module = LoadModule(file);
                if (module != null)
                {
                    modules.Add(module);
                }
            }
            catch (Exception ex)
            {
                _logger.LogError(ex, $"Erro ao carregar assembly: {file}");
            }
        }

        return modules;
    }
}
```

```

public IModule LoadModule(string assemblyPath)
{
    _logger.LogDebug($"Carregando assembly: {assemblyPath}");

    // Carregar assembly
    var assembly = Assembly.LoadFrom(assemblyPath);

    // Procurar tipos que implementam IModule
    var moduleTypes = assembly.GetTypes()
        .Where(t => typeof(IModule).IsAssignableFrom(t) && !t.IsInterface && !t.IsAbstract);

    foreach (var type in moduleTypes)
    {
        try
        {
            var module = (IModule)Activator.CreateInstance(type);
            _logger.LogInformation($"Módulo descoberto: {module.Name} v{module.Version}");
            return module;
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, $"Erro ao instanciar módulo: {type.Name}");
        }
    }

    return null;
}
}

```

4.6 Configuração e Uso do Project Factory

csharp

```
// Program.cs
var host = Host.CreateDefaultBuilder(args)
    .ConfigureServices((context, services) =>
    {
        // Registrar infraestrutura do sistema modular
        services.AddSingleton<IModuleRegistry, ModuleRegistry>();
        services.AddSingleton<IModuleLoader, AssemblyModuleLoader>();
        services.AddSingleton<IProjectFactory, ProjectFactory>();

        // Construir o projeto (carregar todos os módulos)
        var serviceProvider = services.BuildServiceProvider();
        var projectFactory = serviceProvider.GetRequiredService<IProjectFactory>();
        projectFactory.BuildProject(services);
    })
    .Build();

// Inicializar módulos após construção do container
var projectFactory = host.Services.GetRequiredService<IProjectFactory>();
projectFactory.InitializeModules(host.Services);

await host.RunAsync();
```

MÓDULO 5: Boas Práticas e Anti-Patterns

5.1 Boas Práticas Essenciais

Prática 1: Mantenha Factories Simples

Factories devem focar apenas em criação e configuração inicial. Lógica de negócio complexa não pertence a factories.

```
csharp
```

```
//  BOM: Factory focado em criação
public class PaymentProcessorFactory
{
    private readonly IServiceProvider _serviceProvider;

    public IPaymentProcessor Create(string type)
    {
        return type switch
        {
            "credit" => _serviceProvider.GetRequiredService<CreditCardProcessor>(),
            "boleto" => _serviceProvider.GetRequiredService<BoletoProcessor>(),
            _ => throw new ArgumentException()
        };
    }
}

//  RUIM: Factory com lógica de negócio
public class BadPaymentProcessorFactory
{
    public IPaymentProcessor Create(Payment payment)
    {
        // Validações complexas não pertencem aqui
        if (payment.Amount <= 0)
            throw new InvalidOperationException();

        // Cálculos de negócio não pertencem aqui
        var fee = CalculateFee(payment);
        payment.TotalAmount = payment.Amount + fee;

        // Persistência não pertence aqui
        SavePayment(payment);

        return new CreditCardProcessor();
    }
}
```

Prática 2: Use Interfaces para Factories

Sempre defina interfaces para factories, permitindo testes e substituição:

csharp


```
//  BOM: Interface permite substituição e testes
public interface INotificationFactory
{
    INotifier Create(string type);
}

public class NotificationFactoryTests
{
    [Fact]
    public void Service_UsesFactory_Correctly()
    {
        var mockFactory = new Mock<INotificationFactory>();
        var mockNotifier = new Mock<INotifier>();

        mockFactory.Setup(f => f.Create("email"))
            .Returns(mockNotifier.Object);

        var service = new NotificationService(mockFactory.Object);
        // ...teste
    }
}
```

Prática 3: Registre Factories com Lifetime Adequado

```
csharp

// Factories geralmente são Singleton ou Transient
// Singleton: quando a factory não mantém estado e é thread-safe
services.AddSingleton<IPaymentFactory, PaymentFactory>();

// Transient: quando a factory precisa de dependências Scoped
services.AddTransient<IReportFactory, ReportFactory>();

// NUNCA Scoped para factories - não faz sentido semântico
```

Prática 4: Valide Entradas

```
csharp
```

```
//  BOM: Validação clara e exceções específicas
public IPaymentProcessor Create(string type)
{
    if (string.IsNullOrEmpty(type))
    {
        throw new ArgumentException(
            "Tipo de pagamento não pode ser vazio",
            nameof(type));
    }

    return type.ToLower() switch
    {
        "credit" => _serviceProvider.GetRequiredService<CreditCardProcessor>(),
        "boleto" => _serviceProvider.GetRequiredService<BoletoProcessor>(),
        "pix" => _serviceProvider.GetRequiredService<PixProcessor>(),
        _ => throw new NotSupportedException(
            $"Tipo de pagamento não suportado: {type}. " +
            $"Tipos válidos: credit, boleto, pix")
    };
}
```

Prática 5: Documente Tipos Suportados

```
csharp
/// <summary>
/// Factory para criação de processadores de pagamento.
/// </summary>
/// <remarks>
/// Tipos de pagamento suportados:
/// - "credit": Cartão de crédito
/// - "boleto": Boleto bancário
/// - "pix": PIX (pagamento instantâneo)
/// </remarks>
public interface IPaymentFactory
{
    /// <summary>
    /// Cria um processador de pagamento do tipo especificado.
    /// </summary>
    /// <param name="type">Tipo do processador (credit, boleto, pix)</param>
    /// <returns>Instância do processador</returns>
    /// <exception cref="ArgumentException">Quando type é nulo ou vazio</exception>
    /// <exception cref="NotSupportedException">Quando tipo não é suportado</exception>
    IPaymentProcessor Create(string type);
}
```

5.2 Anti-Patterns Comuns

Anti-Pattern 1: God Factory

Factory que conhece muitas classes e tem responsabilidades demais:

```
csharp

// ❌ ANTI-PATTERN: God Factory
public class ApplicationFactory
{
    public object Create(string type)
    {
        return type switch
        {
            "payment" => CreatePaymentProcessor(),
            "notification" => CreateNotifier(),
            "report" => CreateReportGenerator(),
            "logger" => CreateLogger(),
            "repository" => CreateRepository(),
            "validator" => CreateValidator(),
            "cache" => CreateCache(),
            // ...50+ tipos diferentes
            _ => throw new ArgumentException()
        };
    }
}

// ✅ SOLUÇÃO: Múltiplas factories especializadas
public interface IPaymentFactory { }
public interface INotificationFactory { }
public interface IReportFactory { }
// Cada uma com responsabilidade única
```

Anti-Pattern 2: Factory com Estado Mutável

// ❌ *ANTI-PATTERN: Factory com estado mutável compartilhado*

```
public class StatefulFactory
{
    private int _counter = 0; // Estado compartilhado = problema

    public IProcessor Create()
    {
        _counter++; // Não thread-safe
        return new Processor(_counter);
    }
}
```

// ✅ *SOLUÇÃO: Factories devem ser stateless ou thread-safe*

```
public class StatelessFactory
{
    public IProcessor Create()
    {
        // Cria objetos sem depender de estado interno
        return new Processor(Guid.NewGuid());
    }
}
```

Anti-Pattern 3: Factory que Retorna Null

csharp

// ❌ *ANTI-PATTERN: Retornar null força verificações em todo código cliente*

```
public IPaymentProcessor Create(string type)
{
    return type switch
    {
        "credit" => new CreditCardProcessor(),
        _ => null // NUNCA faça isso
    };
}
```

// ✅ *SOLUÇÃO: Sempre lance exceção para casos inválidos*

```
public IPaymentProcessor Create(string type)
{
    return type switch
    {
        "credit" => new CreditCardProcessor(),
        _ => throw new NotSupportedException($"Tipo inválido: {type}")
    };
}
```

// ✅ *ALTERNATIVA: Null Object Pattern (se fizer sentido)*

```
public IPaymentProcessor Create(string type)
{
    return type switch
    {
        "credit" => new CreditCardProcessor(),
        _ => new NullPaymentProcessor() // Implementação vazia que não faz nada
    };
}
```

Anti-Pattern 4: Tight Coupling entre Factory e Implementações

csharp

// ❌ *ANTI-PATTERN: Factory cria objetos com new*

```
public class TightlyCoupledFactory
```

```
{  
    public INotifier Create(string type)  
    {  
        return type switch  
        {  
            "email" => new EmailNotifier("smtp.server.com", 587), // Acoplamento forte  
            "sms" => new SmsNotifier("api-key-hardcoded"),  
            _ => throw new ArgumentException()  
        };  
    }  
}
```

// ✅ *SOLUÇÃO: Use DI container*

```
public class DecoupledFactory
```

```
{  
    private readonly IServiceProvider _serviceProvider;  
  
    public INotifier Create(string type)  
    {  
        return type switch  
        {  
            "email" => _serviceProvider.GetRequiredService<EmailNotifier>(),  
            "sms" => _serviceProvider.GetRequiredService<SmsNotifier>(),  
            _ => throw new ArgumentException()  
        };  
    }  
}
```

Anti-Pattern 5: Factory Method Excessivamente Complexo

csharp

// ❌ ANTI-PATTERN: Lógica complexa de decisão no factory method

```
public IProcessor Create(ProcessingRequest request)
{
    if (request.Type == "A")
    {
        if (request.Priority == High)
        {
            if (request.Size > 1000)
            {
                return new LargeHighPriorityTypeAProcessor();
            }
            else
            {
                return new SmallHighPriorityTypeAProcessor();
            }
        }
        else
        {
            // ...mais aninhamento
        }
    }
    // ...várias páginas de ifs aninhados
}
```

// ✅ SOLUÇÃO: Strategy Pattern + Configuration

```
public class ProcessorFactory
{
    private readonly Dictionary<ProcessorKey, Type> _mappings;

    public ProcessorFactory()
    {
        _mappings = new Dictionary<ProcessorKey, Type>
        {
            [new ProcessorKey("A", Priority.High, SizeRange.Large)] = typeof(LargeHighPriorityTypeAProcessor),
            [new ProcessorKey("A", Priority.High, SizeRange.Small)] = typeof(SmallHighPriorityTypeAProcessor),
            // ...configuração clara
        };
    }

    public IProcessor Create(ProcessingRequest request)
    {
        var key = new ProcessorKey(request.Type, request.Priority, GetSizeRange(request.Size));
        if (_mappings.TryGetValue(key, out var type))
        {
            return (IProcessor)_serviceProvider.GetRequiredService(type);
        }
    }
}
```

```
        throw new NotSupportedException($"Configuração não suportada: {key}");  
    }  
}
```

5.3 SOLID Principles e Factories

Single Responsibility Principle (SRP)

Uma factory deve ter apenas uma razão para mudar: mudanças nos tipos que ela cria.

```
csharp  
  
// ✅ BOM: Factory com responsabilidade única  
public class PaymentProcessorFactory : IPaymentProcessorFactory  
{  
    // Única responsabilidade: criar processadores de pagamento  
    public IPaymentProcessor Create(string type) { }  
}  
  
// ❌ RUIM: Factory com múltiplas responsabilidades  
public class BadFactory  
{  
    public IPaymentProcessor CreateProcessor() { }  
    public void ValidatePayment(Payment p) { } // Validação não é criação  
    public void LogTransaction(Transaction t) { } // Logging não é criação  
}
```

Open/Closed Principle (OCP)

Factories devem ser abertas para extensão mas fechadas para modificação:

```
csharp
```



```
// ✅ BOM: Extensível sem modificação
public class ExtensibleNotificationFactory
{
    private readonly Dictionary<string, Func<INotifier>> _creators;

    public ExtensibleNotificationFactory()
    {
        _creators = new Dictionary<string, Func<INotifier>>();
    }

    // Permite adicionar novos tipos sem modificar a classe
    public void Register(string type, Func<INotifier> creator)
    {
        _creators[type] = creator;
    }

    public INotifier Create(string type)
    {
        if (_creators.TryGetValue(type, out var creator))
        {
            return creator();
        }
        throw new NotSupportedException($"Tipo não registrado: {type}");
    }
}

// Uso
factory.Register("email", () => new EmailNotifier());
factory.Register("sms", () => new SmsNotifier());
// Novos tipos podem ser adicionados externamente
```

Liskov Substitution Principle (LSP)

Todas as implementações de uma factory devem ser substituíveis:

```
csharp
```

//  BOM: Todas as implementações respeitam o contrato

```
public interface INotificationFactory
```

```
{  
    INotifier Create(string type);  
}
```

// Todas lançam exceção para tipos inválidos (comportamento consistente)

```
public class EmailOnlyFactory : INotificationFactory
```

```
{  
    public INotifier Create(string type)  
    {  
        if (type != "email")  
            throw new NotSupportedException();  
        return new EmailNotifier();  
    }  
}
```

```
public class FullNotificationFactory : INotificationFactory
```

```
{  
    public INotifier Create(string type)  
    {  
        return type switch  
        {  
            "email" => new EmailNotifier(),  
            "sms" => new SmsNotifier(),  
            _ => throw new NotSupportedException()  
        };  
    }  
}
```

Interface Segregation Principle (ISP)

Interfaces de factory devem ser coesas e específicas:

csharp

```
// ✅ BOM: Interfaces segregadas
public interface IPaymentProcessorFactory
{
    IPaymentProcessor CreateProcessor(string type);
}

public interface IPaymentValidatorFactory
{
    IPaymentValidator CreateValidator(string type);
}

// ❌ RUIM: Interface monolítica
public interface IBadPaymentFactory
{
    IPaymentProcessor CreateProcessor(string type);
    IPaymentValidator CreateValidator(string type);
    IPaymentLogger CreateLogger(string type);
    IPaymentNotifier CreateNotifier(string type);
    // Clientes são forçados a depender de métodos que não usam
}
```

Dependency Inversion Principle (DIP)

Tanto código de alto nível quanto factories devem depender de abstrações:

```
csharp
```

```
// ✅ BOM: Todos dependem de abstrações
public interface IPaymentProcessor { }
public interface IPaymentFactory
{
    IPaymentProcessor Create(string type);
}

public class OrderService // Alto nível
{
    private readonly IPaymentFactory _factory; // Depende de abstração

    public OrderService(IPaymentFactory factory)
    {
        _factory = factory;
    }
}

public class PaymentFactory : IPaymentFactory // Baixo nível
{
    // Também depende de abstrações (IServiceProvider)
    public IPaymentProcessor Create(string type) { }
}
```

MÓDULO 6: Testes e Verificação

6.1 Estratégias de Teste para Factories

Teste 1: Criação Básica

```
csharp
```

```

public class PaymentFactoryTests
{
    [Fact]
    public void Create_WithValidType_ReturnsCorrectImplementation()
    {
        // Arrange
        var services = new ServiceCollection();
        services.AddTransient<CreditCardProcessor>();
        services.AddTransient<BoletoProcessor>();
        var provider = services.BuildServiceProvider();
        var factory = new PaymentFactory(provider);

        // Act
        var processor = factory.Create("credit");

        // Assert
        Assert.NotNull(processor);
        Assert.IsType<CreditCardProcessor>(processor);
    }

    [Theory]
    [InlineData("credit")]
    [InlineData("boleto")]
    [InlineData("pix")]
    public void Create_WithAllValidTypes_Succeeds(string type)
    {
        // Arrange
        var factory = CreateConfiguredFactory();

        // Act
        var processor = factory.Create(type);

        // Assert
        Assert.NotNull(processor);
        Assert.IsAssignableFrom<IPaymentProcessor>(processor);
    }
}

```

Teste 2: Tratamento de Erros

csharp

```

public class PaymentFactoryErrorTests
{
    [Fact]
    public void Create_WithInvalidType_ThrowsNotSupportedException()
    {
        // Arrange
        var factory = CreateConfiguredFactory();

        // Act & Assert
        var exception = Assert.Throws<NotSupportedException>(() =>
        {
            factory.Create("invalid-type");
        });

        Assert.Contains("invalid-type", exception.Message);
    }

    [Theory]
    [InlineData(null)]
    [InlineData("")]
    [InlineData(" ")]
    public void Create_WithInvalidInput_ThrowsArgumentException(string type)
    {
        // Arrange
        var factory = CreateConfiguredFactory();

        // Act & Assert
        Assert.Throws<ArgumentException>(() =>
        {
            factory.Create(type);
        });
    }
}

```

Teste 3: Integração com DI

csharp

```

public class PaymentFactoryIntegrationTests
{
    [Fact]
    public void Factory_ResolvedProcessorWithDependencies()
    {
        // Arrange
        var services = new ServiceCollection();

        // Registrar dependências dos processadores
        services.AddSingleton<ILogger<CreditCardProcessor>>(
            Mock.Of<ILogger<CreditCardProcessor>>());
        services.AddSingleton<IConfiguration>(
            Mock.Of<IConfiguration>());

        // Registrar processador com dependências
        services.AddTransient<CreditCardProcessor>();

        // Registrar factory
        services.AddSingleton<IPaymentFactory, PaymentFactory>();

        var provider = services.BuildServiceProvider();
        var factory = provider.GetRequiredService<IPaymentFactory>();

        // Act
        var processor = factory.Create("credit");

        // Assert
        Assert.NotNull(processor);
        Assert.IsType<CreditCardProcessor>(processor);
        // Verificar que dependências foram injetadas corretamente
    }
}

```

Teste 4: Lifecycle Behavior

csharp

```

public class FactoryLifecycleTests
{
    [Fact]
    public void Factory_RegisteredAsTransient_CreatesNewInstanceEachTime()
    {
        // Arrange
        var services = new ServiceCollection();
        services.AddTransient<CreditCardProcessor>();
        services.AddSingleton<IPaymentFactory, PaymentFactory>();

        var provider = services.BuildServiceProvider();
        var factory = provider.GetRequiredService<IPaymentFactory>();

        // Act
        var processor1 = factory.Create("credit");
        var processor2 = factory.Create("credit");

        // Assert
        Assert.NotSame(processor1, processor2);
    }

    [Fact]
    public void Factory_RegisteredAsSingleton_ReturnsConsistentResults()
    {
        // Arrange
        var services = new ServiceCollection();
        services.AddTransient<CreditCardProcessor>();
        services.AddSingleton<IPaymentFactory, PaymentFactory>();

        var provider = services.BuildServiceProvider();

        // Act
        var factory1 = provider.GetRequiredService<IPaymentFactory>();
        var factory2 = provider.GetRequiredService<IPaymentFactory>();

        // Assert
        Assert.Same(factory1, factory2);
    }
}

```

6.2 Testes de Abstract Factory

csharp


```
public class ReportFactoryTests
{
    [Fact]
    public void PdfFactory_CreatesConsistentFamily()
    {
        // Arrange
        var factory = CreatePdfReportFactory();

        // Act
        var header = factory.CreateHeader();
        var table = factory.CreateTable();
        var chart = factory.CreateChart();

        // Assert
        Assert.IsType<PdfReportHeader>(header);
        Assert.IsType<PdfReportTable>(table);
        Assert.IsType<PdfReportChart>(chart);
    }

    [Fact]
    public void ReportGenerator_WorksWithDifferentFactories()
    {
        // Arrange
        var pdfFactory = CreatePdfReportFactory();
        var excelFactory = CreateExcelReportFactory();

        var testData = new[]
        {
            new[] { "A", "B" },
            new[] { "C", "D" }
        };
        var chartData = new[] { 1.0, 2.0, 3.0 };

        // Act & Assert - PDF
        var pdfGenerator = new ReportGenerator(pdfFactory);
        var pdfException = Record.Exception(() =>
        {
            pdfGenerator.GenerateReport("Test", testData, chartData);
        });
        Assert.Null(pdfException);

        // Act & Assert - Excel
        var excelGenerator = new ReportGenerator(excelFactory);
        var excelException = Record.Exception(() =>
        {
            excelGenerator.GenerateReport("Test", testData, chartData);
```

```
});  
Assert.Null(excelException);  
}  
}
```

6.3 Testes de Project Factory

csharp

```
public class ProjectFactoryTests
{
    [Fact]
    public void BuildProject_LoadsAllModules()
    {
        // Arrange
        var services = new ServiceCollection();
        var mockRegistry = new Mock<IModuleRegistry>();
        var mockLoader = new Mock<IModuleLoader>();

        var testModules = new List<IModule>
        {
            new TestModuleA(),
            new TestModuleB()
        };

        mockLoader.Setup(l => l.DiscoverModules())
            .Returns(testModules);

        services.AddSingleton(mockRegistry.Object);
        services.AddSingleton(mockLoader.Object);
        services.AddSingleton<ILogger<ProjectFactory>>>(
            Mock.Of<ILogger<ProjectFactory>>>());

        var factory = new ProjectFactory(
            mockRegistry.Object,
            mockLoader.Object,
            Mock.Of<ILogger<ProjectFactory>>>());

        // Act
        factory.BuildProject(services);

        // Assert
        mockRegistry.Verify(r => r.Register(It.IsAny<IModule>()), Times.Exactly(2));
    }

    [Fact]
    public void BuildProject_RespectsModuleDependencies()
    {
        // Arrange
        var moduleB = new Mock<IModule>();
        moduleB.Setup(m => m.Name).Returns("ModuleB");
        moduleB.Setup(m => m.GetType())
            .Returns(typeof(TestModuleB));

        var moduleA = new Mock<IModule>();
```

```

moduleA.Setup(m => m.Name).Returns("ModuleA");
moduleA.Setup(m => m.GetType())
    .Returns(typeof(TestModuleA));

// ModuleA depende de ModuleB
var modules = new[] { moduleA.Object, moduleB.Object };

var loadOrder = new List<string>();

moduleA.Setup(m => m.ConfigureServices(It.IsAny<IServiceCollection>()))
    .Callback(() => loadOrder.Add("ModuleA"));

moduleB.Setup(m => m.ConfigureServices(It.IsAny<IServiceCollection>()))
    .Callback(() => loadOrder.Add("ModuleB"));

// Act
// (executar ProjectFactory com esses módulos)

// Assert
Assert.Equal("ModuleB", loadOrder[0]); // ModuleB deve carregar primeiro
Assert.Equal("ModuleA", loadOrder[1]);
}
}

[Module(Name = "ModuleA", Dependencies = new[] { "ModuleB" })]
public class TestModuleA : IModule
{
    public string Name => "ModuleA";
    public string Version => "1.0";
    public string Description => "Test Module A";
    public void ConfigureServices(IServiceCollection services) { }
    public void Initialize(IServiceProvider serviceProvider) { }
}

[Module(Name = "ModuleB")]
public class TestModuleB : IModule
{
    public string Name => "ModuleB";
    public string Version => "1.0";
    public string Description => "Test Module B";
    public void ConfigureServices(IServiceCollection services) { }
    public void Initialize(IServiceProvider serviceProvider) { }
}

```

6.4 Mocks e Stubs para Factories

csharp

// Criando mock de factory para testes

```
public class ServiceUsingFactoryTests
{
    [Fact]
    public void Service_UsesFactory_Correctly()
    {
        // Arrange
        var mockFactory = new Mock<IPaymentFactory>();
        var mockProcessor = new Mock<IPaymentProcessor>();

        mockFactory.Setup(f => f.Create("credit"))
            .Returns(mockProcessor.Object);

        mockProcessor.Setup(p => p.Process(It.IsAny<Payment>()))
            .Returns(new ProcessingResult { Success = true });

        var service = new PaymentService(mockFactory.Object);

        // Act
        var result = service.ProcessPayment(new Payment
        {
            Type = "credit",
            Amount = 100.00m
        });

        // Assert
        Assert.True(result.Success);
        mockFactory.Verify(f => f.Create("credit"), Times.Once);
        mockProcessor.Verify(p => p.Process(It.IsAny<Payment>()), Times.Once);
    }
}
```

// Stub factory para testes

```
public class StubPaymentFactory : IPaymentFactory
{
    private readonly IPaymentProcessor _processor;

    public StubPaymentFactory(IPaymentProcessor processor)
    {
        _processor = processor;
    }

    public IPaymentProcessor Create(string type)
    {
        return _processor; // Sempre retorna a mesma instância
    }
}
```

```
}  
}
```

MÓDULO 7: Casos Reais e Trade-offs

7.1 Caso Real: Sistema de Notificações

Contexto: Aplicação e-commerce precisa enviar notificações por múltiplos canais (email, SMS, push) baseado em preferências do usuário.

Desafios:

- Cada canal tem configuração diferente
- Alguns canais têm rate limits
- Fallback quando canal primário falha
- Tracking de envios e custos

Solução com Factory Pattern:

```
csharp
```

// Interface do notificador

```
public interface INotifier
{
    Task<NotificationResult> SendAsync(NotificationMessage message);
    bool SupportsChannel(NotificationChannel channel);
}
```

// Factory com metadados

```
public class NotificationFactory : INotificationFactory
{
    private readonly IServiceProvider _serviceProvider;
    private readonly Dictionary<NotificationChannel, NotifierMetadata> _metadata;

    public NotificationFactory(IServiceProvider serviceProvider)
    {
        _serviceProvider = serviceProvider;
        _metadata = new Dictionary<NotificationChannel, NotifierMetadata>
        {
            [NotificationChannel.Email] = new NotifierMetadata
            {
                Type = typeof(EmailNotifier),
                MaxRetries = 3,
                CostPerMessage = 0.01m
            },
            [NotificationChannel.Sms] = new NotifierMetadata
            {
                Type = typeof(SmsNotifier),
                MaxRetries = 2,
                CostPerMessage = 0.15m
            },
            [NotificationChannel.Push] = new NotifierMetadata
            {
                Type = typeof(PushNotifier),
                MaxRetries = 1,
                CostPerMessage = 0.00m
            }
        };
    }

    public INotifier Create(NotificationChannel channel)
    {
        if (!_metadata.TryGetValue(channel, out var metadata))
        {
            throw new NotSupportedException($"Canal não suportado: {channel}");
        }
    }
}
```



```

        return (INotifier)_serviceProvider.GetRequiredService(metadata.Type);
    }

    public NotifierMetadata GetMetadata(NotificationChannel channel)
    {
        return _metadata.TryGetValue(channel, out var metadata)
            ? metadata
            : null;
    }

    public IEnumerable<NotificationChannel> GetAvailableChannels()
    {
        return _metadata.Keys;
    }
}

// Serviço orquestrador com fallback
public class NotificationService
{
    private readonly INotificationFactory _factory;
    private readonly ILogger<NotificationService> _logger;

    public async Task<NotificationResult> SendWithFallbackAsync(
        NotificationMessage message,
        NotificationChannel[] preferredChannels)
    {
        foreach (var channel in preferredChannels)
        {
            try
            {
                var notifier = _factory.Create(channel);
                var result = await notifier.SendAsync(message);

                if (result.Success)
                {
                    _logger.LogInformation(
                        $"Notificação enviada via {channel} para {message.Recipient}");
                    return result;
                }

                _logger.LogWarning(
                    $"Falha ao enviar via {channel}: {result.ErrorMessage}");
            }
            catch (Exception ex)
            {
                _logger.LogError(ex,
                    $"Erro ao tentar enviar via {channel}");
            }
        }
    }
}

```

```
    }  
    }  
  
    return NotificationResult.Failed("Todos os canais falharam");  
    }  
}
```

Trade-offs desta solução:

✅ Vantagens:

- Fácil adicionar novos canais sem modificar código existente
- Metadados centralizados facilitam gerenciamento
- Fallback automático aumenta confiabilidade
- Testável - cada notificador pode ser testado isoladamente

⚠️ Desvantagens:

- Overhead de abstração para casos simples
- Metadados podem ficar desatualizados
- Difícil implementar lógica de seleção muito complexa

7.2 Caso Real: Sistema de Pagamentos

Contexto: Plataforma de marketplace com múltiplos métodos de pagamento e necessidade de cálculos de taxa específicos por método.

Desafios:

- Cada processador tem validações diferentes
- Taxas variam por método e valor
- Compliance e auditoria
- Integração com gateways externos

Solução com Abstract Factory:

csharp

// Família de componentes de pagamento

public interface IPaymentProcessor

```
{  
    Task<PaymentResult> ProcessAsync(PaymentRequest request);  
}
```

public interface IPaymentValidator

```
{  
    ValidationResult Validate(PaymentRequest request);  
}
```

public interface IFeeCalculator

```
{  
    decimal CalculateFee(decimal amount);  
}
```

// Abstract Factory

public interface IPaymentComponentFactory

```
{  
    IPaymentProcessor CreateProcessor();  
    IPaymentValidator CreateValidator();  
    IFeeCalculator CreateFeeCalculator();  
}
```

// Concrete Factory - Cartão de Crédito

public class CreditCardComponentFactory : IPaymentComponentFactory

```
{  
    private readonly IServiceProvider _serviceProvider;  
  
    public CreditCardComponentFactory(IServiceProvider serviceProvider)  
    {  
        _serviceProvider = serviceProvider;  
    }  
  
    public IPaymentProcessor CreateProcessor()  
    => _serviceProvider.GetRequiredService<CreditCardProcessor>();  
  
    public IPaymentValidator CreateValidator()  
    => _serviceProvider.GetRequiredService<CreditCardValidator>();  
  
    public IFeeCalculator CreateFeeCalculator()  
    => new CreditCardFeeCalculator(percentageFee: 2.5m, fixedFee: 0.30m);  
}
```

// Serviço de pagamento que usa a factory

public class PaymentService

```

{
    private readonly Func<string, IPaymentComponentFactory> _factorySelector;
    private readonly ILogger<PaymentService> _logger;

    public PaymentService(
        Func<string, IPaymentComponentFactory> factorySelector,
        ILogger<PaymentService> logger)
    {
        _factorySelector = factorySelector;
        _logger = logger;
    }

    public async Task<PaymentResult> ProcessPaymentAsync(PaymentRequest request)
    {
        // Selecionar factory apropriada
        var factory = _factorySelector(request.PaymentMethod);

        // Criar família completa de componentes
        var validator = factory.CreateValidator();
        var feeCalculator = factory.CreateFeeCalculator();
        var processor = factory.CreateProcessor();

        // Validar
        var validationResult = validator.Validate(request);
        if (!validationResult.IsValid)
        {
            return PaymentResult.Failed(validationResult.Errors);
        }

        // Calcular taxa
        var fee = feeCalculator.CalculateFee(request.Amount);
        request.Fee = fee;
        request.TotalAmount = request.Amount + fee;

        // Processar
        _logger.LogInformation(
            $"Processando pagamento de {request.TotalAmount:C} " +
            $"via {request.PaymentMethod}");

        return await processor.ProcessAsync(request);
    }
}

// Configuração no Program.cs
services.AddTransient<CreditCardProcessor>();
services.AddTransient<CreditCardValidator>();
services.AddTransient<CreditCardComponentFactory>();

```

```
services.AddTransient<BoletoProcessor>();
services.AddTransient<BoletoValidator>();
services.AddTransient<BoletoComponentFactory>();

services.AddSingleton<Func<string, IPaymentComponentFactory>>(provider =>
{
    return method => method.ToLower() switch
    {
        "credit" => provider.GetRequiredService<CreditCardComponentFactory>(),
        "boleto" => provider.GetRequiredService<BoletoComponentFactory>(),
        _ => throw new NotSupportedException($"Método não suportado: {method}")
    };
});
```

Trade-offs:

✅ Vantagens:

- Garante que componentes compatíveis sejam usados juntos
- Facilita adição de novos métodos de pagamento
- Lógica de negócio encapsulada em componentes específicos
- Fácil testar cada método de pagamento isoladamente

⚠️ Desvantagens:

- Mais classes e interfaces para manter
- Pode ser over-engineering para poucos métodos de pagamento
- Curva de aprendizado para novos desenvolvedores

7.3 Performance: Quando Otimizar

Cenário 1: Factory Invocado Frequentemente

csharp

// ❌ PROBLEMA: Factory cria nova instância a cada requisição

```
public class UnoptimizedReportFactory
{
    public IReportGenerator Create(string format)
    {
        return format switch
        {
            "pdf" => new PdfReportGenerator(), // Cria novo objeto
            "excel" => new ExcelReportGenerator(),
            _ => throw new NotSupportedException()
        };
    }
}
```

// ✅ SOLUÇÃO: Cache de instâncias stateless

```
public class OptimizedReportFactory
{
    private readonly ConcurrentDictionary<string, IReportGenerator> _cache;

    public OptimizedReportFactory()
    {
        _cache = new ConcurrentDictionary<string, IReportGenerator>();
    }

    public IReportGenerator Create(string format)
    {
        return _cache.GetOrAdd(format.ToLower(), key =>
        {
            return key switch
            {
                "pdf" => new PdfReportGenerator(),
                "excel" => new ExcelReportGenerator(),
                _ => throw new NotSupportedException()
            };
        });
    }
}
```

Quando aplicar cache:

- Objetos são stateless (sem estado)
- Thread-safe
- Criação é cara (inicialização complexa)
- Factory é chamado muito frequentemente

Quando NÃO aplicar cache:

- Objetos mantêm estado mutável
- Cada chamada deve retornar instância nova
- Objetos consomem muita memória

Cenário 2: Lazy Loading de Dependências Pesadas

```
csharp
// ✅ Factory com lazy loading
public class OptimizedPaymentFactory
{
    private readonly IServiceProvider _serviceProvider;
    private readonly Lazy<CreditCardProcessor> _creditCardProcessor;
    private readonly Lazy<BoletoProcessor> _boletoProcessor;

    public OptimizedPaymentFactory(IServiceProvider serviceProvider)
    {
        _serviceProvider = serviceProvider;

        // Lazy loading - só cria quando necessário
        _creditCardProcessor = new Lazy<CreditCardProcessor>(
            () => _serviceProvider.GetRequiredService<CreditCardProcessor>());

        _boletoProcessor = new Lazy<BoletoProcessor>(
            () => _serviceProvider.GetRequiredService<BoletoProcessor>());
    }

    public IPaymentProcessor Create(string type)
    {
        return type switch
        {
            "credit" => _creditCardProcessor.Value, // Só cria na primeira chamada
            "boleto" => _boletoProcessor.Value,
            _ => throw new NotSupportedException()
        };
    }
}
```

7.4 Trade-off: Simplicidade vs Extensibilidade

Quando usar Simple Factory:

- ✅ Conjunto fixo e pequeno de tipos (2-5)

- ☒ Sem necessidade de extensibilidade futura
- ☒ Prototipagem rápida
- ☒ Lógica de criação simples

Quando usar Factory Method:

- ☒ Necessidade de subclasses customizarem criação
- ☒ Template Method pattern se beneficia
- ☒ Lógica de criação complexa que varia
- ☒ Boa extensibilidade sem modificar código existente

Quando usar Abstract Factory:

- ☒ Múltiplas famílias de produtos relacionados
- ☒ Necessidade de garantir compatibilidade entre produtos
- ☒ Troca de família inteira deve ser fácil
- ☒ Consistência entre produtos é crítica

Quando usar Project Factory (Modular):

- ☒ Sistema com plugins ou módulos dinâmicos
- ☒ Arquitetura de microkernel
- ☒ Necessidade de descoberta e registro automático
- ☒ Gerenciamento complexo de dependências entre módulos

MÓDULO 8: Checklist e Materiais de Apoio

8.1 Checklist de Code Review para Factories

Design e Arquitetura:

- ☐ Factory tem interface bem definida?
- ☐ Responsabilidade única (SRP) é respeitada?
- ☐ Factory está aberto para extensão mas fechado para modificação (OCP)?
- ☐ Não há violações de princípios SOLID?
- ☐ Padrão escolhido é apropriado para o problema?

Implementação:

- ☐ Factory valida parâmetros de entrada?
- ☐ Exceções apropriadas são lançadas para casos inválidos?
- ☐ Factory nunca retorna null (ou usa Null Object Pattern intencionalmente)?
- ☐ Não há código de negócio complexo dentro do factory?
- ☐ Factory é stateless ou thread-safe?

Integração com DI:

- ☐ Factory está registrado com lifetime apropriado?
- ☐ Dependências são injetadas via construtor (não Service Locator anti-pattern)?
- ☐ Uso de `IServiceProvider` é justificado e limitado?
- ☐ Objetos criados têm suas dependências resolvidas corretamente?

Testabilidade:

- ☐ Factory tem interface que permite mocking?
- ☐ Testes cobrem casos de sucesso?
- ☐ Testes cobrem casos de erro?
- ☐ Testes verificam integração com DI?

Documentação:

- ☐ Tipos suportados estão documentados?
- ☐ Exceções possíveis estão documentadas?
- ☐ Exemplos de uso estão presentes?
- ☐ Decisões de design estão registradas?

Performance:

- ☐ Factory não cria objetos desnecessariamente?
- ☐ Cache é usado quando apropriado?
- ☐ Lazy loading é aplicado se criação é cara?

8.2 Glossário de Termos

Abstract Factory: Padrão que fornece interface para criar famílias de objetos relacionados sem especificar classes concretas.

Captive Dependency: Situação onde serviço de vida curta (Transient/Scoped) é capturado por serviço de vida longa (Singleton), causando problemas.

Concrete Factory: Implementação específica de uma factory que cria tipos concretos.

Constructor Injection: Técnica de DI onde dependências são fornecidas via construtor da classe.

Dependency Injection (DI): Padrão onde dependências são fornecidas externamente em vez de criadas internamente.

Factory Method: Padrão que define interface para criar objetos, permitindo subclasses decidir qual classe instanciar.

God Factory: Anti-pattern onde factory conhece e cria muitos tipos diferentes, violando SRP.

Inversion of Control (IoC): Princípio onde controle de fluxo é invertido - framework chama seu código.

IServiceCollection: Interface do .NET para registro de serviços em container de DI.

IServiceProvider: Interface do .NET para resolução de serviços registrados.

Lifetime: Período de vida de um objeto gerenciado por container (Transient, Scoped, Singleton).

Module: Unidade lógica de funcionalidade que pode ser habilitada/desabilitada independentemente.

Null Object Pattern: Padrão onde objeto "vazio" implementa interface mas não faz nada, evitando verificações de null.

Project Factory: Factory arquitetural que orquestra criação e registro de módulos em aplicação modular.

Scoped: Lifetime onde mesma instância é compartilhada dentro de um escopo (ex: requisição HTTP).

Service Locator: Anti-pattern onde serviço solicita dependências diretamente do container.

Simple Factory: Técnica (não padrão GoF) de método estático que encapsula criação de objetos.

Singleton: Lifetime onde única instância existe para toda vida da aplicação.

Transient: Lifetime onde nova instância é criada a cada resolução.

8.3 Recursos Adicionais

Documentação Oficial Microsoft:

- [Dependency injection in .NET](#)
- [.NET Generic Host](#)
- [Service lifetimes](#)

Livros Recomendados:

- "Design Patterns: Elements of Reusable Object-Oriented Software" - Gang of Four
- "Dependency Injection Principles, Practices, and Patterns" - Steven van Deursen & Mark Seemann
- "Clean Architecture" - Robert C. Martin

Artigos e Blogs:

- Martin Fowler - Inversion of Control Containers and Dependency Injection Pattern

- Mark Seemann - Blog sobre DI e design patterns
- Microsoft .NET Blog - Melhores práticas

8.4 Exercícios Práticos Sugeridos

Exercício Básico: Sistema de Logging Implementar factory para diferentes destinos de log (Console, Arquivo, Database) com DI.

Exercício Intermediário: Sistema de Relatórios Implementar Abstract Factory para gerar relatórios em diferentes formatos (PDF, Excel, HTML) mantendo componentes consistentes.

Exercício Avançado: Sistema Modular de E-commerce Implementar Project Factory que gerencia módulos (Catálogo, Carrinho, Pagamento, Notificação) com dependências entre eles, suportando habilitação/desabilitação dinâmica.

Conclusão

Este material apresentou uma jornada completa desde os fundamentos de Dependency Injection até padrões avançados de Factory em aplicações .NET empresariais. A integração entre DI e Factory Patterns é essencial no desenvolvimento moderno, permitindo criar aplicações extensíveis, testáveis e que respeitam princípios SOLID.

Os conceitos apresentados devem ser praticados através dos exercícios e exemplos de código fornecidos. Lembre-se: padrões são ferramentas, não regras absolutas. Use-os quando agregarem valor, não por obrigação.

Próximos Passos:

1. Revisar exemplos de código práticos
 2. Implementar exercícios propostos
 3. Estudar diagramas UML fornecidos
 4. Aplicar conceitos em projetos reais
 5. Compartilhar conhecimento com a equipe
-

Versão do Documento: 1.0

Última Atualização: Dezembro 2024

Autor: Material preparado para ensino de Padrões de Design em .NET

Licença: Material educacional para uso acadêmico